

Planning guide for Fetcho automatic driver assignment

I need you to think about this as an operations system, not just an algorithm.

Goal:

Build a dispatch system for Podgorica that is reliable, profitable, and simple enough to operate at launch, but structured so it can improve later.

Main business goals:

- deliver on time
- reduce driver waiting
- reduce restaurant waiting
- avoid cold food
- allow safe batching only when it helps
- maximize completed deliveries per driver
- keep fuel and wasted driving low
- be easy to debug and override

This should not start with a super advanced optimization engine.

It should start conservative, explainable, and measurable.

1. Operating assumptions

Context:

- starting in Podgorica
- city is relatively small
- many orders will likely be short distance
- launch may include around 20 restaurants and 10 to 15 drivers
- scooters and cars are likely the main vehicle types
- if batching is too aggressive, quality will drop very fast

Main principle:

Reliability is more important than theoretical maximum efficiency.

If a decision increases the chance of lateness, it should usually be rejected.

2. Core model

For every order, the system should predict three moments:

1. predictedReadyAt
2. predictedPickupAt
3. predictedDeliveredAt

The dispatch system should constantly try to keep these predictions realistic.

For every driver, the system should know:

- current location
- last location update time
- vehicle type
- status
- active order count
- current assigned route if any

3. System components

The system should be separated into clear parts.

A. Prep time estimator

Predict when the restaurant will have the order ready.

Inputs:

- branch average prep time
- menu item complexity
- current restaurant load
- safety buffer

Suggested formula:

predictedReadyAt =

createdAt

- branch avg prep
- item adjustment
- load adjustment
- small safety buffer

B. Route ETA engine

Use routing time, not air distance.

Need:

- driver to restaurant ETA
- restaurant to customer ETA

This should ideally come from Google Maps or a similar routing source.

C. Candidate driver finder

Given an order, find drivers who are actually eligible.

D. Driver scoring engine

Given candidates, score them and rank them.

E. Assign or hold engine

Decide whether to assign now or wait.

F. Batching evaluator

Only used when the system already supports limited batching.

G. Risk evaluator

Mark orders as low, medium, or high risk.

4. Driver states

Define driver operational states clearly.

At minimum:

- offline
- online_available
- holding
- assigned_to_pickup
- waiting_at_restaurant
- delivering
- temporarily_unavailable

Important:

Do not assign from vague or unclear states.

Status transitions must be strict.

5. Order states relevant for dispatch

Restaurant status flow can stay simple:

- pending
- accepted
- ready

Dispatch-relevant internal values:

- predictedReadyAt
- assignedDriverId
- dispatchStatus
- riskLevel

Optional dispatchStatus:

- not_ready_for_dispatch
- eligible_for_dispatch
- held
- assigned

6. Assign or hold logic

This is one of the most important parts.

You should not assign every order immediately.

Reason:

If food will only be ready in 18 minutes and nearest driver is 2 minutes away, early assignment wastes driver time and reduces throughput.

Recommended idea:

Use a dispatch window.

Example:

- if order is more than 10 minutes away from predictedReadyAt, hold
- if order is 4 to 10 minutes away, evaluate drivers
- if order is under 4 minutes away, assign more aggressively

This is not fixed forever, but it is a good launch rule.

Hold means:

- do not assign yet
- order remains monitored
- retry evaluation every 30 to 60 seconds
- or retry on important events like driver becoming available

Assign means:

- choose the best valid driver
- place a short lock on that driver
- confirm assignment
- transition driver to assigned state

7. Race condition protection

This must be handled properly or the whole system becomes unreliable.

Main race condition:

Two orders may try to assign the same driver at the same time.

Protection needed:

- atomic claim or lock
- short-lived holding state
- transaction around assignment if possible
- recheck driver availability before final write

Recommended pattern:

1. scoring engine picks best driver
2. system attempts atomic hold/claim
3. if claim succeeds, continue assignment
4. if claim fails, recalculate next best driver

Do not rely on frontend or timing luck.

Important:

Batching decisions must also be protected from race conditions.

A driver cannot be made eligible for a second order while another assignment is already modifying that route state unless properly locked.

8. Candidate driver filtering

Before scoring, reduce the candidate pool.

Filter by:

- same city
- online_available
- recent location update
- not overloaded
- correct delivery mode
- within a reasonable radius or ETA threshold

Optional helper layer:

- same or nearby operational zone

Important:

Zones should be helpers, not final truth.

Routing ETA is the real decision layer.

9. Driver scoring

Do not make the first version magical.

Make it explainable.

Lower score should be better.

Scoring should penalize:

- long travel time to restaurant
- arriving too late to ready time
- arriving far too early and waiting too long
- already carrying load
- batching detour
- likely lateness
- stale location
- zone mismatch if useful

Simple penalty style is enough.

Example structure:

score =

travel_to_restaurant_penalty

- late_arrival_penalty
- excessive_wait_penalty
- active_load_penalty
- detour_penalty
- risk_penalty

Very important:

Predicted lateness should either heavily penalize or reject the candidate entirely.

10. ETA calculation

Total ETA shown to customer should be based on:

prep remaining

- driver to restaurant
- restaurant to customer
- small delivery buffer

There should be two modes:

A. no driver assigned yet

Use estimated driver arrival assumptions

B. driver assigned

Use actual assigned driver route ETA

Update ETA when:

- restaurant accepts
- driver assigned
- restaurant marks ready
- driver picks up
- route conditions change significantly

11. Batching strategy

Do not start with aggressive batching.

Best launch model:

maximum 2 orders per driver

Allow batching only if all are true:

- same branch
- same city
- same or nearby operational zone
- ready times are close
- additional route detour is small
- first customer is not harmed significantly
- second order still remains safe
- driver capacity allows it

Hard rule:

If batching creates a realistic chance of lateness, reject batching.

Recommended launch thresholds:

- max 2 orders
- ready-time gap ≤ 6 minutes
- extra delay to first customer ≤ 5 minutes
- extra total route time ≤ 8 minutes

No cross-restaurant batching at launch.

That is where complexity grows too fast.

12. Profit logic

The goal is not only speed.

It is profitable speed.

Every delivery brings revenue, but bad dispatch increases:

- fuel cost
- idle time
- late refunds or support issues
- driver inefficiency

- restaurant frustration

Most profitable behavior at launch:

- reduce empty driving
- reduce waiting at restaurant
- avoid risky long detours
- batch only when safe
- keep drivers in useful areas
- avoid accepting dispatch choices that reduce total future throughput

Important business idea:

Sometimes not batching makes more money because it keeps one driver free sooner for the next nearby order.

The system should prefer:

- high confidence on-time deliveries
- low wasted movement
- stable throughput

not:

- forcing one driver to carry too much just because the map looked efficient

13. Podgorica-specific thinking

Podgorica is small enough that zones can be useful operationally, but routing time should still be primary.

Good use of zones:

- grouping drivers
- narrowing candidates
- batching hints
- overload control
- analytics

Bad use of zones:

- treating zones as the final pricing or ETA truth

Correct rule:

coordinates and route ETA are truth

zones are structure

Because Podgorica is compact:

- assignment can be more aggressive
- batching can work well if controlled
- distance thresholds can be smaller and stricter
- same-zone and neighboring-zone logic is practical

14. Risk system

Every order should have a risk level.

Suggested levels:

- low
- medium
- high

Risk should consider:

- restaurant delay vs predictedReadyAt
- no driver assigned close to ready time
- assigned driver too far
- predictedDeliveredAt beyond target
- batching side effects
- stale driver information

Purpose:

- ops/admin visibility
- future dashboard prioritization
- future overload behavior
- easier debugging

15. Overload modes

If demand spikes, the system needs behavior modes.

Suggested modes:

- normal
- busy
- overload

Normal:

standard dispatch rules

Busy:

- larger buffers
- less willingness to batch
- stronger lateness penalties

Overload:

- very conservative assignment
- disable risky batching
- extend ETA shown to users
- protect current orders instead of blindly accepting all opportunities

This matters because a strong launch can fail if the system keeps accepting risky dispatch choices under pressure.

16. Data to store

On order:

- predictedReadyAt
- predictedPickupAt
- predictedDeliveredAt

- actualReadyAt
- actualPickedUpAt
- actualDeliveredAt
- assignedDriverId
- riskLevel
- dispatchStatus
- batchGroupId optional

On driver:

- currentLat
- currentLng
- currentCityId
- currentZoneId
- lastLocationAt
- vehicleType
- status
- maxSimultaneousOrders
- activeOrderCount

On branch:

- cityId
- zoneId
- avgPrepMinutes
- prepBufferMinutes
- lat
- lng

17. Recommended implementation order

Do not build all at once.

Phase 1

- driver live location and status
- prep prediction
- route ETA abstraction

- candidate filtering
- driver scoring
- assign or hold
- race condition protection

Phase 2

- risk levels
- limited batching max 2
- same branch only
- safe delay simulation

Phase 3

- overload modes
- zone-aware tuning
- historical accuracy improvements

18. What should be measurable

You should design this so performance can be measured.

Track:

- average time driver waits at restaurant
- average time order waits after ready
- predicted vs actual ready time
- predicted vs actual delivered time
- late delivery rate
- average driver idle time
- batching acceptance rate
- batching success rate
- number of reassigned orders
- number of high-risk orders

Without this, the system cannot improve.

19. What not to do at launch

Do not start with:

- complicated multi-restaurant batching
- three-order routes
- AI prediction hype
- black-box scoring nobody understands
- dispatch rules that are impossible to override
- too many factors at once

Launch system should be:

- strict
- explainable
- measurable
- safe

20. Final design philosophy

This system should optimize for trust first, then efficiency.

The right launch rule is:

If efficiency creates a real chance of lateness, choose reliability.

That will protect:

- customer satisfaction
- restaurant confidence
- driver efficiency
- long-term profitability

The smartest system at the start is not the most complicated one.

It is the one that keeps deliveries predictable and scalable.